

Public Safety Alerts Aggregator & Explorer

Course: Software Development 1 -ITC303 / ITC306

Assessment item 2 - Group Project : Requirements

Baseline & Technology Decision Pack

Due Date: May 15, 2026

Author: Critical Signal

Institute: Charles Sturt University

Instructor: Jason Howarth

Ettore Vescio

tv00047@gmail.com

11799221

Thomas Smith

thomas1134smith@gmail.com

11787500

Christian Finocchiaro

cajfino@gmail.com

11854560

Pradip Pandey

pradippandey555@gmail.com

11831987

Subject Code	ITC303 / ITC306
Assignment	Requirements Baseline & Technology Decision Pack
Team name	Critical Signal
Project Title	2026-16 – Public Safety Alerts Aggregator & Explorer
Team members	Ettore Vescio, Christian Finocchiaro, Thomas Smith, Pradip Pandey
Submission date	May 15, 2026

Section 1 – Requirements Baseline

1a. Requirements List (Prioritised)

#	Requirement / User Story	Priority (MoSCoW)
R01	As a user, I want an intuitive user interface to view real-time public safety alerts on a map so that I can understand incidents near me without getting lost or confused.	Must
R02	As a system, I want to ingest alert data from external RSS/API sources (e.g. NSW RFS, NSW SES, TFNSW) so that alerts are automatically retrieved and existing alerts are updated instead of duplicated.	Must
R03	As a user, I want to filter alerts by region, council area, category, and severity and I want the page to remember my filter preferences so that I can see relevant alerts as soon as I open the page.	Must
R04	As a user, I want to subscribe to alert filters using my email so that I receive notifications when relevant alerts occur.	Must
R05	As a system, I want to send notification emails when alerts match user subscriptions so that users are informed promptly.	Must
R06	As a user, I want to click an alert and view details including source link, status, and location so that I can verify the information.	Should
R07	As a system, I want to store spatial data (marker points, polylines and polygons) so that alerts can be visualised accurately on a map.	Should
R08	As a user, I want a platform that can be easily accessed anywhere at anytime with little to no latency or downtime and clear responsiveness.	Must

1b. Non-Functional Requirements

Performance Requirements

The system must process and display new alerts within 60 seconds of retrieval from external sources. The system must support at least 500 concurrent users with response times under 3 seconds.

Security Requirements

All data in transit must be encrypted using HTTPS (TLS 1.2+). User email data must be securely stored and protected against unauthorised access. Input validation must prevent injection attacks.

Usability Requirements

The interface must be intuitive and usable by non-technical users of all abilities on all devices and screen sizes, with clear map-based visualisation and simple filtering options.

Reliability / Availability

The system must achieve 99% uptime and handle failures in external APIs gracefully without crashing and provide clear intuitive error codes and messages when issues occur.

Compliance / Privacy

The system must comply with the Australian Privacy Act 1988. User email addresses must only be used for alert notifications and not shared with third parties. All data utilised by the system must, at all times, comply with Australian copyright laws as outlined by the Copyright Act 1968. As the data used by the system is third party, the system will not hold copyright on any alerts data, only the source code.

Other Non-Functional Requirements

The system must be scalable to support additional alert sources across Australia. The system must be maintainable with modular architecture and documented APIs.

1c. Acceptance Criteria

Req#	Acceptance Criterion	How to Test / Verify
R01	Alerts are displayed on a map with markers and polygons / polylines	Load the system and verify alerts appear visually on the map with correct coordinates
R02	System successfully retrieves and stores alerts from at least one external RSS/API source and repeats periodically	Run ingestion process and verify alerts are stored in database with correct fields then wait for the predetermined period of time to observe if the process repeats and updates the database
R03	Users can filter alerts by region, council area, category, and severity	Apply filters and verify only matching alerts are displayed
R04	Users can create a subscription using email and selected filters	Submit subscription form and verify record is stored in database
R05	Notification email is sent when a new alert matches a subscription	Trigger matching alert and confirm email is received

1d. Constraints and Dependencies

Constraints	External Dependencies
Project must be completed within university timeline (limited development time)	Availability and reliability of external alert sources (NSW RFS, NSW SES, TFNSW)
Limited hosting budget (prefer free-tier/ cloud solutions)	Internet connectivity for API/RSS retrieval
System must use technologies familiar to the team (Node.js, MySQL)	Email service provider for sending notifications (e.g., SMTP or third-party API)
Alerts from different sources may result in compatibility issues with the existing front/ backend	Third party software libraries, packages and APIs
Large number of alerts need to be handled and stored leading to potentially response delays and storage constraints	Storing user emails is subject to the Australian Privacy Act and Spam Act 2003
	Each data source has a different terms of service and access can be revoked at any time

Additional notes on constraints / dependencies

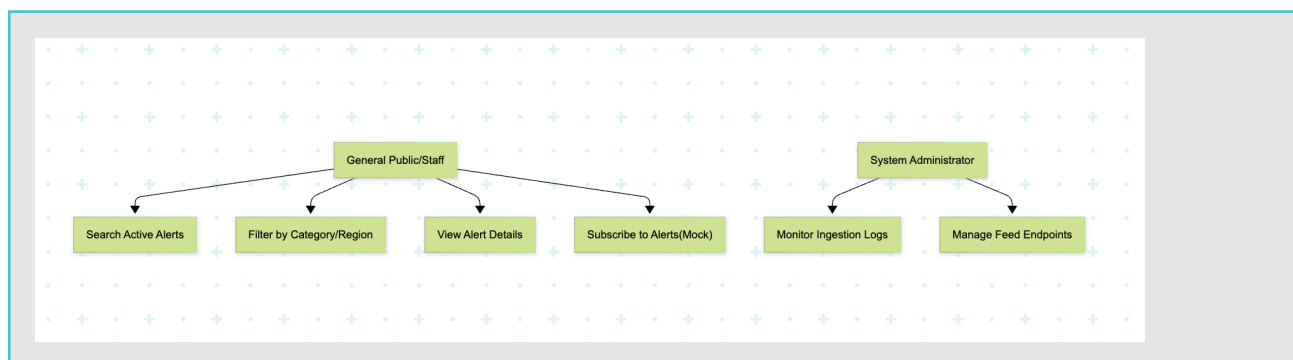
1. Strict time constraint of two academic sessions (Session 1 and 2) i.e. features need to be priority based.
2. Absolutely free and open-source software stack needed; no money to spend on high-quality APIs or managed database selections.
3. The stability of the public APIs and RSS Feeds (TFNSW, NSW RFS); rate limits by these government bodies.
4. Availability of the host environment (e.g., student VM or cloud tiers constraints on compute power).

Section 2 — Models and Diagrams**2a. Project Type**

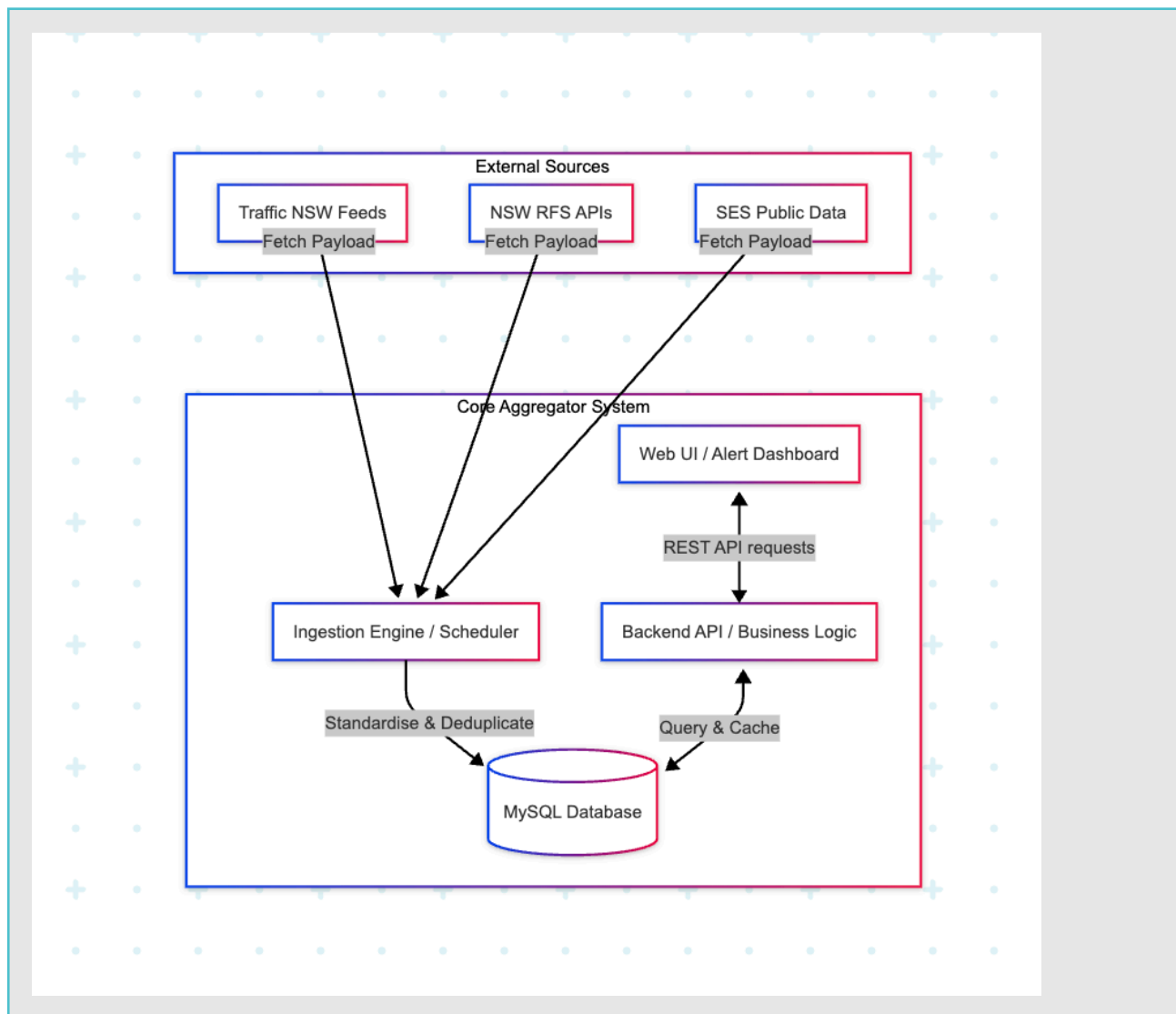
X Software Project

2b. Diagrams

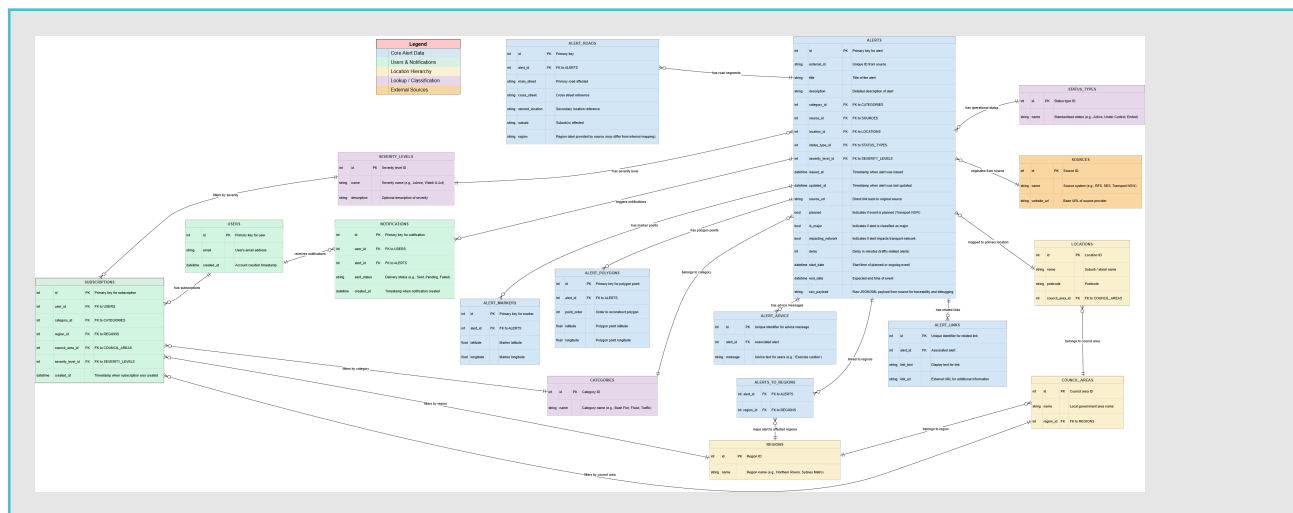
Diagram 1 - Title:

Use Cases**Brief description / key points for Diagram 1**

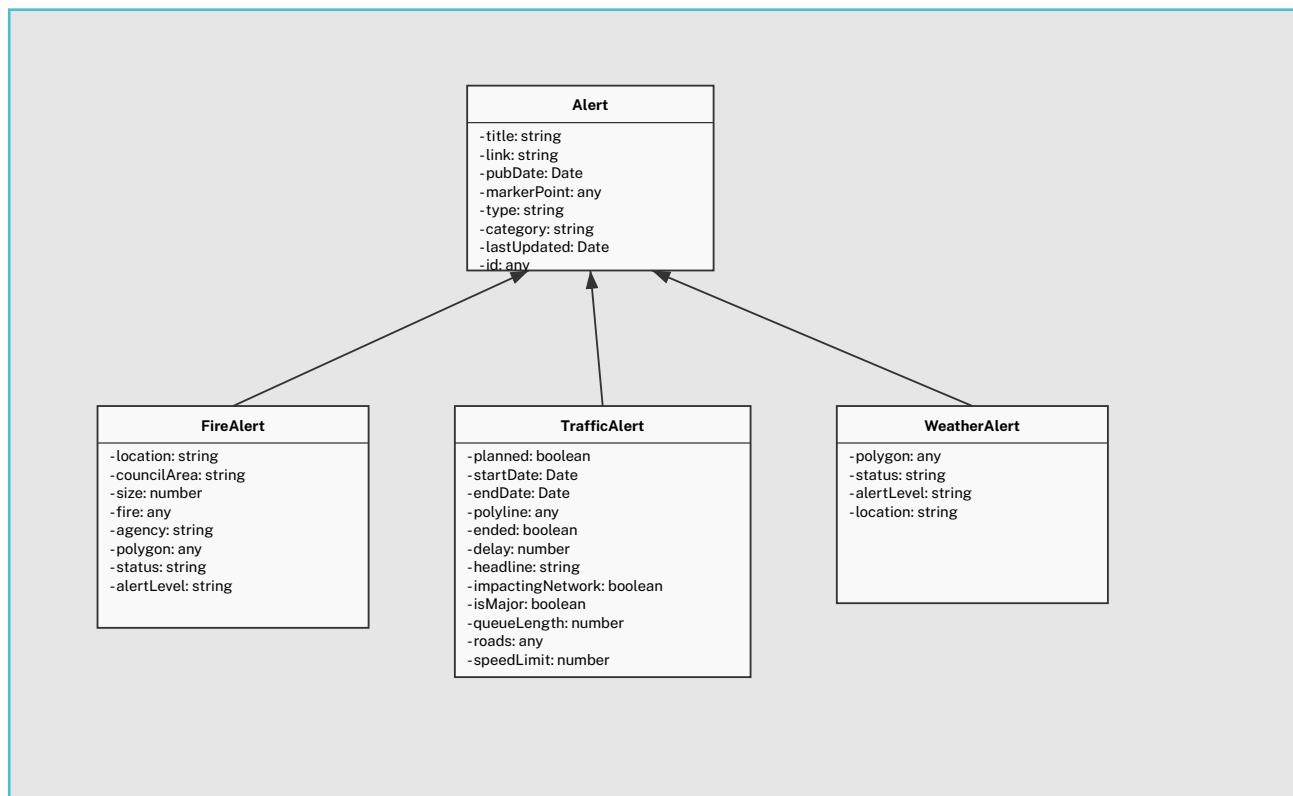
Separates the Interaction between the normal users:
- General Public, (searching and viewing alerts) and system administration (managing the background pipeline and feeds).

Diagram 2 - Title:**System Architecture****Brief description / key points for Diagram 2**

This diagram shows the macro architecture. Data is processed by a scheduled Ingestion Engine which queries external sources and standardises the data and stores it in the relational database. This data is then accessed by the Web interface using the Backend API easily.

Diagram 3 - Title:**Database ERD****Brief description / key points for Diagram 3**

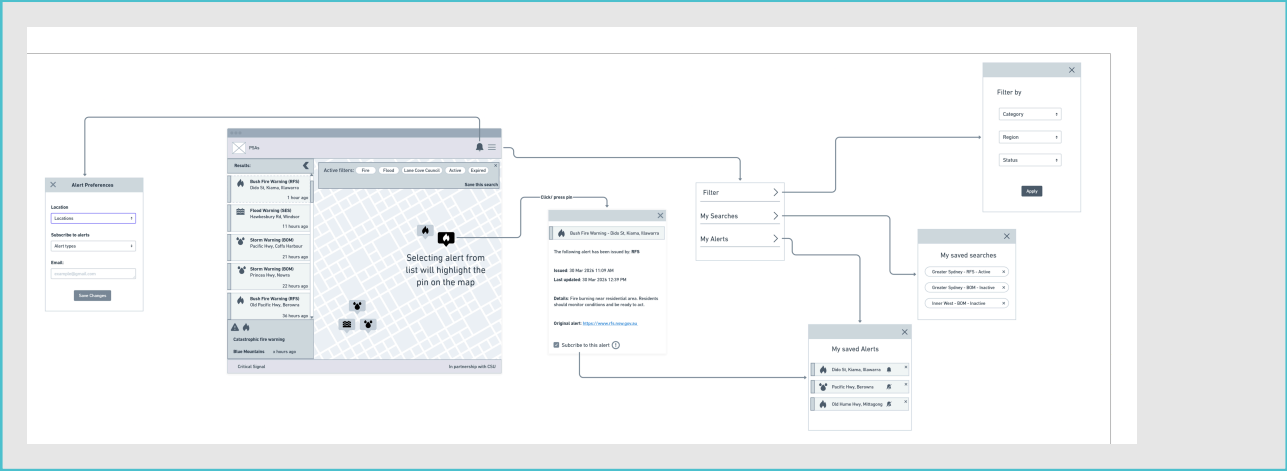
- The schema is designed to support heterogeneous alert feeds while maintaining a standardised core alert structure.
- ALERTS stores standardised alert data from multiple external sources.
- REGIONS, COUNCIL_AREAS, and LOCATIONS support geographic filtering.
- ALERTS_TO_REGIONS allows alerts to affect multiple regions.
- ALERT_MARKERS and ALERT_POLYGONS store geospatial map data.
- Source-specific information is handled using dedicated supporting tables.
- Users can subscribe to filtered alerts and receive notifications.

Diagram 4 - Title:**Class Diagram****Brief description / key points for Diagram 4**

This diagram shows the structure and inheritance of the alert class that define the alert objects. It details what fields are required and how they are inherited from the parent.

Diagram 5 - Title:

Wireframe



Brief description / key points for Diagram 5

The main view is an alert list on the left and an interactive map on the right. Selecting an alert highlights the corresponding pin on the map and clicking the pin opens a modal with more detail, a source link, and the option to subscribe to that individual alert. Active filters are displayed in a bar at the top and can be saved as search patterns, accessible later through the My Searches section. The bell icon opens Alert Preferences where users can set broader subscription rules by region and alert type, delivered via email. A pinned warning area at the bottom of the sidebar displays critical alerts when something severe is happening and stays empty otherwise.

Section 3 – Technology Shortlist and Evaluation

3a. Options Considered

Option A – Name / Description	Option A Backend: Node.js (express.js) Frontend: Next.js (React.js) DB: SQLite for local development and testing, with planned migration to MySQL for deployment
Option B – Name / Description	Option B Backend: Python (Django/Flask), Frontend: HTML/JS + Jinja/Templates, DB: PostgreSQL
Option C – Name / Description	Option C Backend: PHP (Laravel) Frontend: Blade Templates DB: MySQL

3b. Decision Matrix / Trade-off Analysis

Evaluation Criterion	Option A	Option B	Option C
Fit to requirements	5 - Strong fit for API ingestion, real-time updates, and relational data and clean, responsive, dynamic UI and UX	4 - Good for structured apps but heavier for real-time ingestion	2 - Poor and complex implementation of certain features, reduced performance, lack of modularity, lack of control
Security	5 - Great Async Handling and strong built in data/JSON handling	5 - Excellent data/JSON manipulation libraries	3 - Adequate, standard backend
Cost	5 - Open-source, low hosting cost	5 - Open-source	5 - Open-source
Team Skills	4 - Team familiarity with JavaScript	3 - Familiar with most of the team	2 - Low familiarity with PHP
Integration Effort	5 - Easy, reliable, well documented and industry standard development with user friendly packages	4 - Fast API dev with DRF/Flask-REST	3 - Slightly heavier overhead
Scalability	4 - Highly scalable, event driven, object and component orientated development	4 - Scales effectively with configuration	3 - Weighty, can scale but resource heavy
Supportability	5 - Wide range of public and commercial packages and ease of custom middleware development and customisability at all levels	5 - Strong community	4 - Laravel has good built-in mitigations
OVERALL SCORE / RANK	33	30	22

3c. Final Recommendation and Justification

Recommended Option:

Option A

Justification:

Option A was selected as it provides the best overall balance between functionality and performance and is most appropriate for the team's current skill level. SQLite will be used during early development to simplify local testing for each team member, while MySQL is planned for the final deployment environment due to better scalability and multi-user support. MySQL was selected as the database for the final release as all alert data is normalized and follows a consistent structure (as demonstrated in the ERD design). SQLite will be utilized during development to provide each team member local access to the database until the team can secure a web-hosting server. For the front-end, Next.js is ideal for a growing UI with many moving parts like modals, and an interactive map that can be organized using React's state management. Next.js also supports dynamic imports necessary for map rendering. For the back-end, Node.js was selected as it supports asynchronous processing, this prevents a slow or unavailable data source from stalling the entire system. The team's familiarity with JavaScript also reduces the risk and time-commitment involved in learning a new technology, such as Django.

Section 4 — Feasibility Evidence**POC 1 - Title:**

Database Ingestion POC Test

4a. What We Tested - POC 1

A proof-of-concept (PoC) was developed to validate the feasibility of ingesting and storing real-world alert data from multiple sources.

A Node.js script was implemented to:

- Read alert data from JSON files (NSW RFS and TFNSW examples)
- Parse and transform the data into a standardised structure
- Insert the data into a MySQL database based on the designed ERD

The PoC specifically tested:

- Ingestion of heterogeneous data formats from different providers
- Storage of geospatial data (marker points, polygons, and polylines)
- Handling of optional and source-specific attributes
- Prevention of duplicate alerts using a unique identifier

4b. Results - POC 1

The PoC was successful and demonstrated that:

- Alert data from both NSW RFS and TFNSW was successfully ingested and stored in the database
- The system correctly handled different geospatial formats:
 - Polyline data (TFNSW)
 - Polygon data (NSW RFS)
- Marker points and spatial data were inserted into their respective tables without errors
- Alerts were uniquely identified using an external identifier, preventing duplicate records
- Re-running the ingestion script confirmed that duplicate alerts were detected and skipped

No major technical issues were encountered. Minor inconsistencies in field structure across sources were resolved through conditional parsing logic.

4c. What We Learned - POC 1

- External alert feeds vary significantly in structure and require flexible parsing logic
- Geospatial data is represented differently across sources (e.g., polygon vs polyline), requiring a normalised storage approach
- Separating spatial data into dedicated tables improves flexibility and supports accurate map visualisation
- Implementing duplicate detection using a unique identifier is essential for maintaining data integrity
- Storing raw source data supports debugging and future schema adjustments

4d. What Changed as a Result - POC 1

- The database design was refined to better support multiple geospatial formats (polygons and polylines)
- The ingestion approach now includes a transformation layer to standardise incoming data before storage
- Additional fields (e.g., planned, delay, start_date, end_date) were validated as necessary for transport-related alerts
- The use of a unique external identifier for alerts was confirmed as critical to prevent duplication
- Confidence in the selected technology stack (Node.js and MySQL) was validated

POC 2 - Title:

Backend API POC Test

4a. What We Tested - POC 2

A proof of concept backend and API was developed to test the process of retrieving datasets from the required external endpoints (NSW RFS, NSW SES, TFNSW), preparing the data for ingestion and having the data accessible via API endpoints. Endpoints were created for fire alerts and traffic alerts. These endpoints ran functions that collected the data and converted them to alert objects and sent them as an API response.

4b. Results - POC 2

The backend functions and API were successful at retrieving the data and sending it to the frontend in a usable JSON object format. Each API endpoint returned JSON containing the array of each alert type (fire, traffic).

The API successfully returned the JSON containing an array of alerts for both NSW RFS and TFNSW that can be remotely called upon and contains the relevant data to display on a Google Map

4c. What We Learned - POC 2

- latency issues caused by an overwhelming number of markers and polygons being loaded at once
- differing data from each data source requires different normalisation for each alert type

4d. What Changed as a Result - POC 2

- Loading data to the map will be done procedurally to reduce load
- Use of marker clusters on the map to reduce graphic load when rendering the map

POC 3 - Title:

Frontend POC Test

4a. What We Tested - POC 3

Created a simple web based front end that connects to a MySQL database via a Node.js Express backend to display normalised alerts as an interactable list. Clicking an alert in the list highlights the corresponding pin on a static map of NSW. The pin can then be clicked to open a modal with further details about the alert. Also put together a static map of NSW with pins as a visual mockup as a proof of concept. This PoC was also used to evaluate the proposed page layout and see whether screen real-estate is being used effectively. The test was built using vanilla HTML, CSS and JS to keep it simple and just prove the whole thing works end to end. Data flows from the database through the backend (Node.js) and into the UI. Next.js is confirmed for the actual build. Researched best method for saving user preferences and filter settings without a server or user account function.

4b. Results - POC 3

The main flow works as alerts can successfully be retrieved from the database, displayed in the alert list and passed to the modal to show the correct information. The static map works visually but not functionally as pins need to be absolutely positioned over the map, which results in incorrect positioning if the map container or window change in size. This is a clear limitation of vanilla HTML/CSS/JS. An interactive map library like leaflet or API like Google is better suited to dynamic elements like a live map. The page layout sufficiently holistically incorporates all of the necessary features and requirements as part of a single UI, while remaining easy to navigate and readable.

4c. What We Learned - POC 3

Setting up a Node.js server and connecting it to MySQL was simpler than expected. Vanilla JS gets difficult to manage quickly once multiple elements need to stay in sync. Positioning pins over a static image confirmed that a proper mapping library is needed as absolute positioning breaks when the container size changes. With only 4 alerts performance was fine but rendering a large number of alerts, pins and modals without state management could become an issue. Browser storage can be used to maintain some user data (like preferences and filter settings) without a server but is not a reliable long-term solution due to high likelihood of data loss. Our system layout was inspired by existing systems, which is tried and tested and fit for purpose.

4d. What Changed as a Result - POC 3

Technology Decisions

- The limitations of positioning pins over a static map demonstrates the need for a mapping library (like leaflet) for the actual build that fits the project constraints while also meeting the requirements
- Next.js and React were confirmed for the actual build as vanilla JS becomes increasingly difficult to maintain even for a small project, due to the lack of structure and state management. In React, selecting an alert automatically updates the card, the pin and the modal without having to manually connect each element together

Overall Conclusion:

Overall, the feasibility testing validated the proposed architecture, confirmed that the selected technologies are suitable for the final prototype, and reduced implementation risk by proving the end-to-end ingestion, storage, API, and frontend workflow.

Section 5 – Updated Project Controls

5a. Milestones / Plan Snapshot (Post-Decision, Rolling-Wave)

Milestone / Deliverable	Key Tasks	Due Date	Owner(s)
Assignment 2 Completion	Finalise requirements, diagrams, and technology justification	15/05/2026	Whole team
Assignment 2 Submission	Final submission via Brightspace	15/05/2026	Ettore Vescio
Sprint 1 (Core Backend)	Database setup, ingestion script, basic API endpoints	22/05/2026	Ettore Vescio Thomas Smith
Sprint 2 (Filtering & UI)	Implement filtering (region/category), basic frontend	29/05/2026	Christian Finocchiaro Pradip Pandey
Sprint 3 (Notifications)	Implement email notifications, subscription logic, and frontend integration	05/06/2026	Whole team
Assignment 4 Submission	Client-Ready solution architecture & demonstration. Final deliverable: Prototype, demonstration of the system.	12/06/2026	Whole team

5b. Risk Register (Updated)

Risk ID	Risk Description	Likelihood	Impact	Mitigation
R01	Inconsistent external data formats	High	High	Implement transformation layer to standardise incoming data; validate fields before insertion
R02	Missing or incomplete Data from Sources	Medium	Medium	Use optional fields and default values; store raw payload for fallback and debugging
R03	Duplicate alerts from Data Feeds	High	Medium	Use external_id to detect and prevent duplicate inserts
R04	Integration complexity across multiple sources	Medium	High	Modular ingestion design; isolate source-specific logic
R05	The users' filter preferences are saved locally in the browser and can be lost if the browser data is reset or a the system is accessed from a different device.	High	Low	Make the user aware that certain actions will lead to loss of preferences and document it as a known constraint
R06	High concurrent database access may reduce system performance	Low	High	Use database connection pooling and query optimisation to handle concurrent requests efficiently
R07	The project will utilize the free version of Google's 'Maps' API, limited to 28500 map loads per month	Low	Low	Increase budget or limit the amount of times that the map is rendered, Leaflet/ OpenStreetMap may be used if Google Maps usage limits become restrictive.
R08	Unauthorised access to user data could theoretically be used to track someone's location to a specific town	Low	High	Restrict database access, encrypt data in transit using HTTPS, and minimise stored personal data to only required email addresses.
R09	Denial of Service attacks	Low	High	Rate limiting

5c. Change Log

Date	Change By	Description of Change	Reason / Impact
19/04/2026	Thomas Smith	Amended the Google Map to incorporate marker clustering	Will reduce graphical load when rendering the map to improve frontend load times
19/04/2026	Whole team	Swapped priorities of traffic and weather alerts to focus on traffic	Difficulty reliably accessing weather data, traffic alerts will be preferred for the initial prototype
20/04/2026	Ettore Vescio	Introduced source-specific tables (ALERT_ROADS, ALERT_ADVICE, ALERT_LINKS)	Allows flexible handling of different data structures (e.g., TFNSW vs NSW RFS)
25/04/2026	Ettore Vescio	Added raw_payload field to ALERTS table	Supports debugging, traceability, and future schema evolution
08/05/2026	Whole team	Use of SQLite for local development	To simplify local testing for each team member in the development stage of the project
08/05/2026	Whole team	Development database strategy updated	SQLite selected for lightweight local development and testing, with MySQL retained as the planned production database for scalability and concurrent access support

Section 6 — Appendix

6a. Team Contributions Summary

Student Name	Sections Contributed To	Summary of Contributions
Ettore Vescio	1-6	-Led the overall coordination and submission of the assignment. -Contributed ideas to and helped refine all team answers to each section. -Designed and implemented the database schema (ERD), ensuring it supports multiple data sources and geospatial data. -Developed the database ingestion proof-of-concept,

		including parsing and storing real-world alert data (NSW RFS and TFNSW) in MySQL. -Contributed to requirements definition, technology evaluation, feasibility analysis, and project planning sections. -Assisted with Team Meeting notes -Reviewed and integrated team contributions into the final document.
Christian Finocchiaro	1-6	-Contributed ideas to and helped refine all team answers to each section. -Wireframe created using whimsical-constraints, requirements, front-end testing, risk register, technology choice and justification. -Creation of vanilla html/css/js prototype and testing the entire core pipeline passing data from MySQL to a front-end via a node.js backend -Assisted with Team Meeting notes
Thomas Smith	1-6	-Contributed ideas to and helped refine all team answers to each section. -Selected our chosen tech stack for the development of the project. -Developed the first iteration of the backend express.js api prototype to collect alert data from external sources (NSW RFS rss feed and TFNSW api) and to process the data to collect the

		data relevant to our use case to populate an alerts objects that can later be ingested into the database. -Developed the first iteration of the prototype for the frontend Google Maps implementation using next.js to display markers and polygons for all alerts retrieved from the backend api. - Assisted with Team Meeting notes
Pradip Pandey	1-6	-Contributed ideas to and helped refine all team answers to each section. -Developed key aggregator features and NFRs supported by references to academic literature. -Created technical architecture and Use Cases/System Artitecture diagrams via Mermaid. -Completed risk mitigation in Section 5. - Assisted with Team Meeting notes

6b. Supporting Evidence

Evidence Item 1 - Label / Description:

Evidence of the database alerts table after data ingestion test

MySQL Workbench interface showing the 'alerts' table. The table structure is as follows:

id	external_id	title	description	category_id	source_id	location_id	status_type_id	severity_level_id	source_id	updated_at	source_url	planned	is_major	impacting_network	delay	start_date	end_date	rev_period
1	212927	CHANGED TRAFFIC CONDITIONS Near Road ...	Traffic	0000	0000	0000	0000	0000	0000	2024-11-01 07:47:00	https://www.britainia.com/incident-details/212927	1	0	1	0	2024-11-01 07:47:00	2024-06-30 23:45:00	[url: 212927, 'id': 'https://www.britainia.com/incident-details/212927', 'rev': '1', 'type': 'Traffic']

Evidence Item 2 - Label / Description:

Evidence of the database alert markers table after data ingestion test

MySQL Workbench interface showing the 'alert_markers' table. The table structure is as follows:

id	alert_id	latitude	longitude
5	1	-33.79134230	150.76985150
6	2	-32.83198602	151.50342178
NULL	NULL	NULL	NULL

Evidence Item 3 - Label / Description:

Evidence of the database alert polygons table after data ingestion test

The screenshot displays the MySQL Workbench interface. The left sidebar shows the 'SCHEMAS' tree with the 'public-safety-alerts-aggregator' database selected. The 'Tables' list under this database includes 'alert_polygons'. The main query window shows a query: `SELECT * FROM `public-safety-alerts-aggregator`.alert_polygons;`. The 'Result Grid' at the bottom displays the data for the 'alert_polygons' table.

	id	alert_id	point_order	latitude	longitude
1	2	0	-32.83021311	151.50228948	
2	2	1	-32.83035588	151.50332692	
3	2	2	-32.83035588	151.50332454	
4	2	3	-32.83054624	151.50400269	
5	2	4	-32.83092695	151.50440720	
6	2	5	-32.83121249	151.50518052	
7	2	6	-32.83142664	151.50562072	
8	2	7	-32.83162889	151.50566831	
9	2	8	-32.83173597	151.50562072	
10	2	9	-32.83230704	151.50514483	
11	2	10	-32.83242601	151.50420494	
12	2	11	-32.83230704	151.50377664	
13	2	12	-32.83214048	151.50355059	
14	2	13	-32.83172407	151.50312229	
15	2	14	-32.83130766	151.50264640	
16	2	15	-32.83101023	151.50228948	
17	2	16	-32.83078418	151.50219430	
18	2	17	-32.83048675	151.50219430	
19	2	18	-32.83021311	151.50228948	
*	NULL	NULL	NULL	NULL	NULL

Evidence Item 4 - Label / Description:

Shows the results of the Database Ingestion POC Tests

```

18 ingestjs U X
tests > JS ingestjs >...
18 async function ingest() {
19   // 2. Insert into ALERTS
20   const [result] = await conn.execute(
21     `INSERT INTO ALERTS
22     (external_id, title, description, issued_at, updated_at, source_url,
23     VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?)`,
24     [
25       alert.id || alert.title,
26       alert.title,
27       alert.type || "",
28       normalise_date(alert.startDate) || normalise_date(alert.pubDate) ||
29       normalise_date(alert.pubDate) || normalise_date(alert.lastUpdated) ||
30       alert.link || "",
31       alert.planned || false,
32       alert.isMajor || false,
33       alert.impactingNetwork || false,
34       alert.delay || 0,
35       normalise_date(alert.startDate),
36       normalise_date(alert.endDate),
37       JSON.stringify(alert)
38     ]
39   );
40   const alertId = result.insertId;
41   console.log("Inserted ALERT ID:", alertId);
42   // 3. Insert marker point
43   if (alert.markerPoint) {
44     let lat, lng;
45     if (Array.isArray(alert.markerPoint)) {
46       lat = alert.markerPoint[0];
47       lng = alert.markerPoint[1];
48     } else {
49       lat = alert.markerPoint.lat;
50       lng = alert.markerPoint.lng;
51     }
52   }
53 }
54 }
55 }
56 }
57 }
58 }
59 }
60 }
61 }
62 }
63 }
64 }
65 }
66 }
67 }
68 }
69 }

1 traffic_alertjson U {} rfs_alertjson U X
tests > data > {} rfs_alertjson > {} polygon > {} 1
1 {
2   "title": "(JOHN RENSHAW DR), BUCHANAN",
3   "link": "https://www.rfs.nsw.gov.au/fire-informa
4   "pubDate": "2026-04-18T19:05:00.000Z",
5   "markerPoint": {
6     "lat": -32.831986024999935,
7     "lng": 151.5034217700009
8   },
9   "type": "Fire",
10  "category": "No Category",
11  "lastUpdated": "2026-04-19T05:05:00.000Z",
12  "id": "655085",
13  "location": "B68 (JOHN RENSHAW DR), BUCHANAN 232
14  "councilArea": "Cessnock",
15  "size": 5,
16  "fire": true,
17  "agency": "Rural Fire Service",
18  "polygon": [
19    {
20      "lat": -32.830213112,
21      "lng": 151.502289477
22    },
23    {
24      "lat": -32.83035588,
25      "lng": 151.50326922
26    },
27    {
28      "lat": -32.83035588,
29      "lng": 151.50324542
30    },
31    {
32      "lat": -32.830546237,
33      "lng": 151.504002688
34    },
35    {
36      "lat": -32.830926951,

```

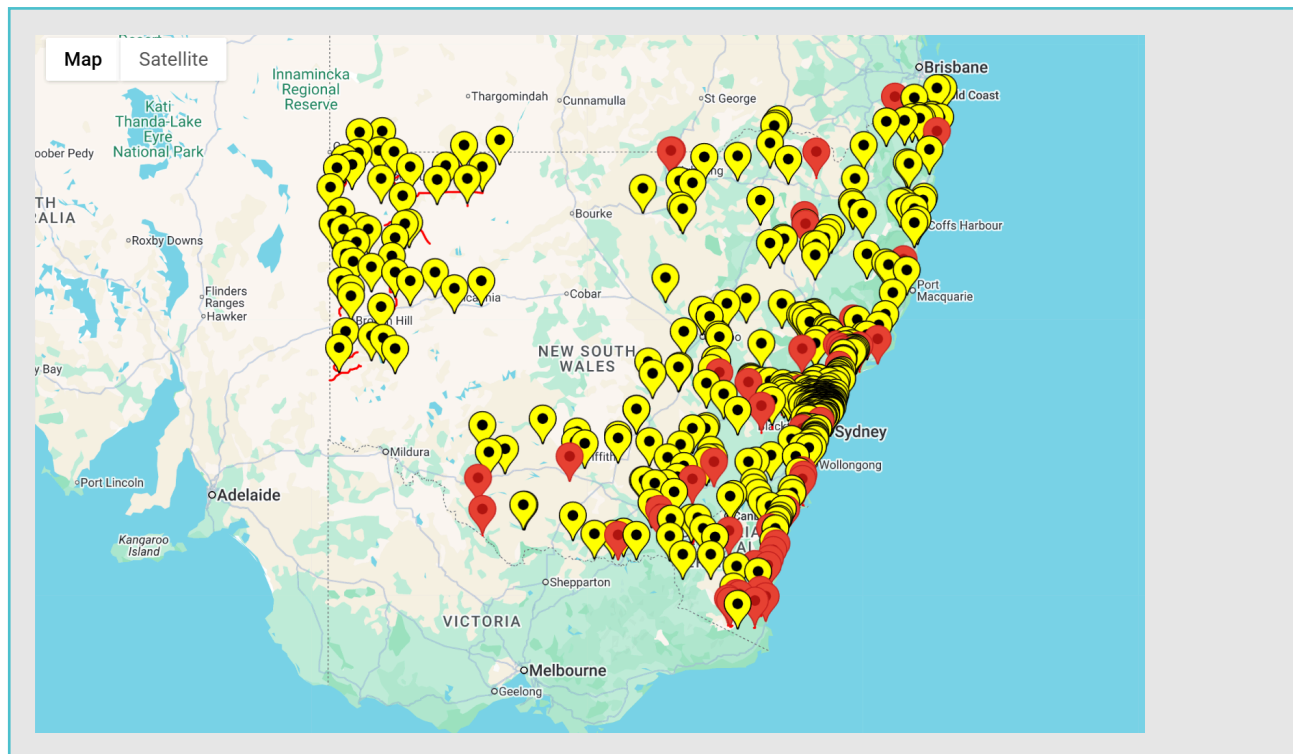
```

PS D:\Programming learning\Uni-Computer_Science\public-safety-alerts-aggregator> node "d:\Programming learning\Uni-Computer_Science\public-safety-alerts-aggregator\tests\ingest.js"
Connected to database
Inserted ALERT ID: 1
Inserted marker
Polyline data found, inserting points...
Inserted poly points
Ingestion complete
PS D:\Programming learning\Uni-Computer_Science\public-safety-alerts-aggregator> node "d:\Programming learning\Uni-Computer_Science\public-safety-alerts-aggregator\tests\ingest.js"
Connected to database
Inserted ALERT ID: 2
Inserted marker
Polygon data found, inserting points...
Inserted poly points
Ingestion complete
PS D:\Programming learning\Uni-Computer_Science\public-safety-alerts-aggregator> node "d:\Programming learning\Uni-Computer_Science\public-safety-alerts-aggregator\tests\ingest.js"
Connected to database
Alert already exists, skipping insert
PS D:\Programming learning\Uni-Computer_Science\public-safety-alerts-aggregator>

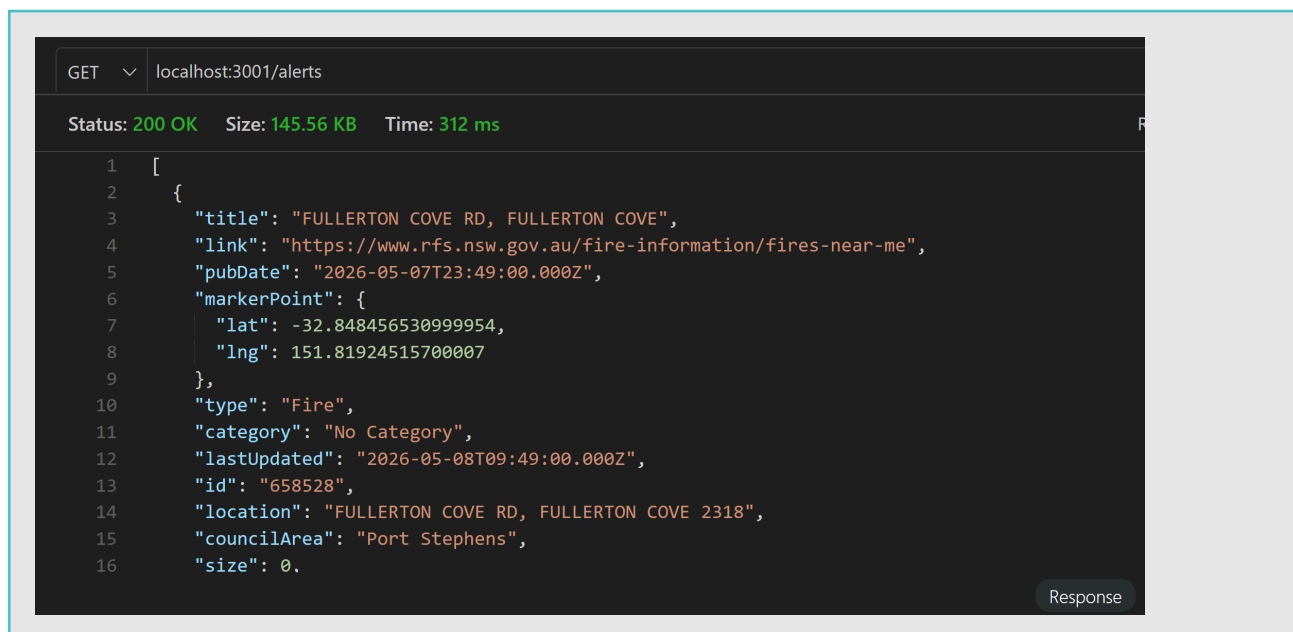
```

Evidence Item 5 - Label / Description:

Shows a google map view of NSW with alert markers pinned on the map

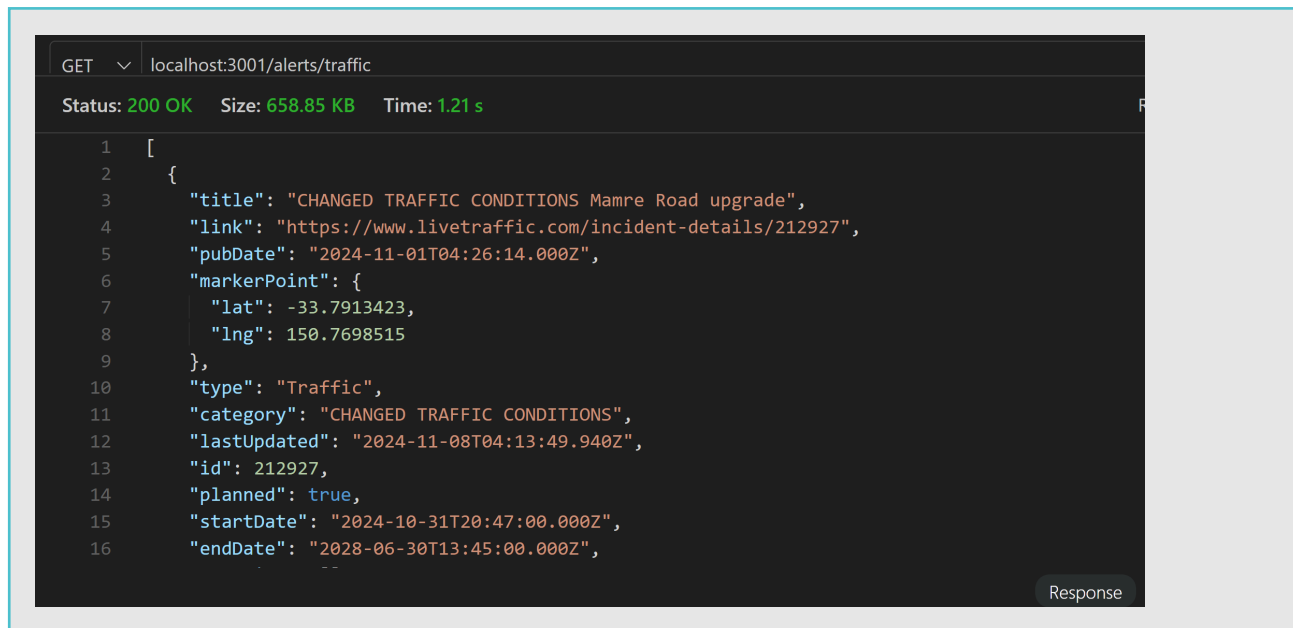
**Evidence Item 6 - Label / Description:**

Shows a sample of the NSW RFS alert ingested into the backend

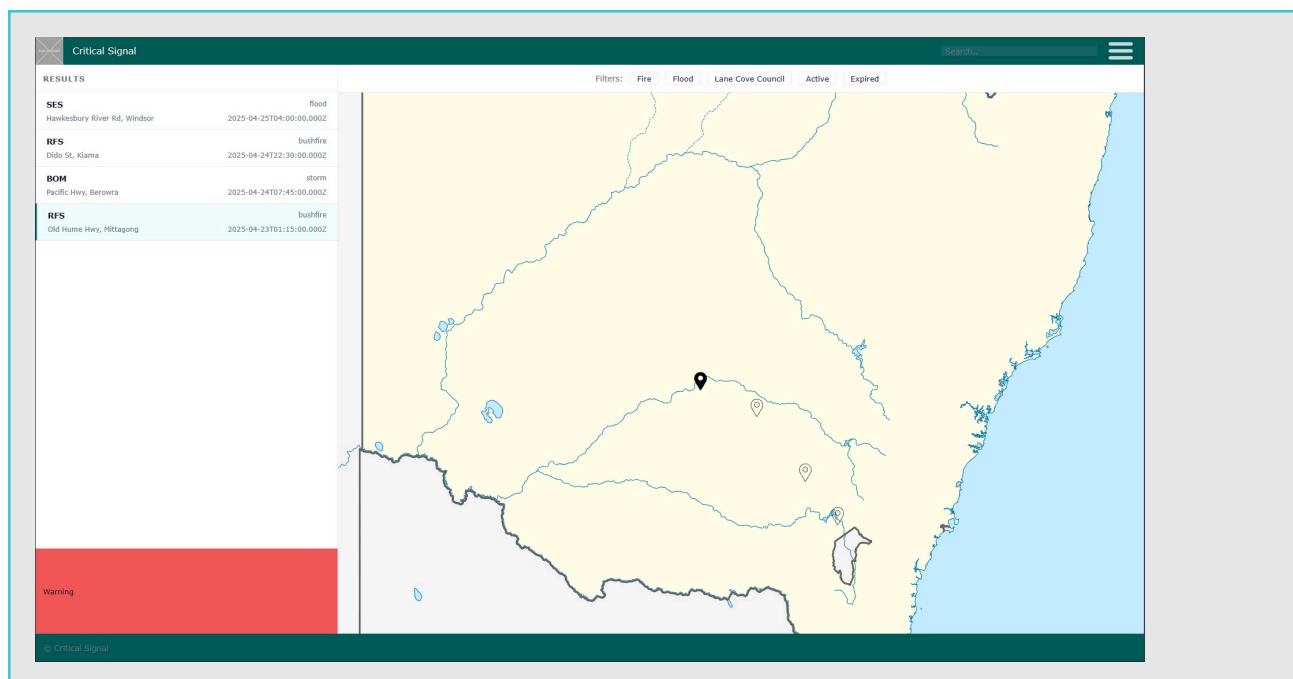


Evidence Item 7 - Label / Description:

Shows a sample of the TFNSW alert ingested into the backend

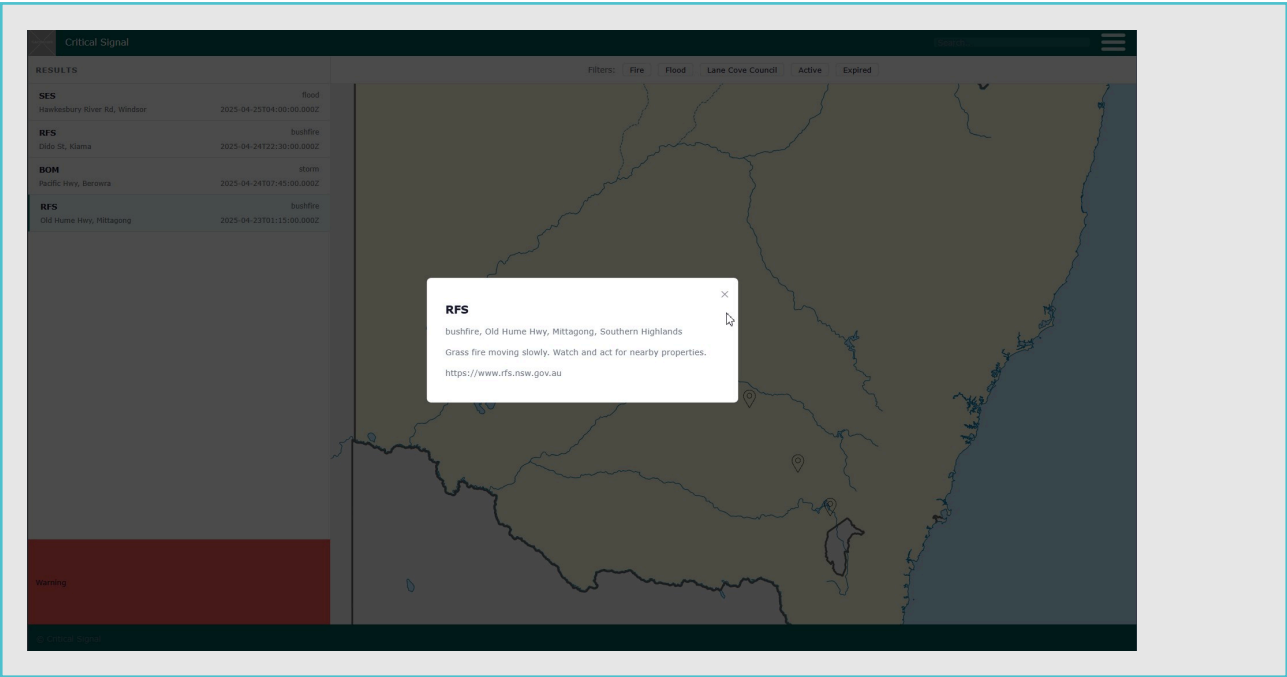
**Evidence Item 8 - Label / Description:**

Shows the result of the frontend POC layout test



Evidence Item 9 - Label / Description:

Shows the result of the frontend POC alert detail view test



Appendix

-.1 /A2-images/meeting-minutes/meeting-05.md

A Meeting 5 – A3 Progress

A.1 Date & Attendees

Date: Sunday, 19 April, 2026

Time: 7:30 PM

Location: Online

Chair: Ettore Vescio

Minutes recorded by: Thomas Smith

Attendees:

- Ettore Vescio (Team Leader)
- Christian Finocchiaro
- Thomas Smith
- Pradip Pandey

A.2 Agenda

1. Recap on current assessment status
2. Collaboratively progress through assessment 2

A.3 Discussion

A.3.1 Update on previous action items

- Confirmed that previously assigned actions items were complete
- Overviewed and discussed assessment 2 progress made by individuals

A.3.2 Alerts update

- Discussed the difficulty finding data sources for weather updates
- Viewed a test implementation of both fire and road alerts
- Discussed the class design of each alert type
- Discussed the implementation of SQL to store the alerts
- Identified some typos in the alert classes
- Discussed difference between polygons and polylines

A.4 Map update

- Confirmed that a map will be included in the prototype of the project
- Viewed a basic test of implementing a map in Next.js
- Identified frontend latency issues due to loading a large number of markers and polygons
- Discussed potential solutions to remedy issue
 - Load only what fits within the bounds of the map
 - Marker clustering to reduce graphical load from markers
 - Sequential loading and updating of data

A.5 Assessment 2 collaboration

- Collectively worked through assessment 2
- Still waiting on wireframes and class diagram to be completed for section 2
- Discussed wireframe progress and UI ideas

- Viewed the data structures from the data sources to better conceptualise the alert class structure to create the class diagram
- Completed sections 1-3
- Reviewed and refined responses to each section

A.6 Open Questions

- Ask Jason about closing assessment 1 to hide Brightspace overdue task alert

A.7 Action Items

Task	Owner	Due
Complete class diagram	Thomas	Due date 26/04/2026
Continue work on wireframes	Christian	Due date 26/04/2026
Find a feasible solution to reduce map lag	Thomas	TBC
Begin sections 4 of assessment 2	Everyone	Due date 26/04/2026

A.8 Next Meeting

Date: Sunday, April 26, 2026

Time: 7:30 PM

A.9 /A2-images/meeting-minutes/meeting-06.md

B Meeting 6 – A2 Work through

B.1 Date & Attendees

Date: Sunday, 26 April, 2026

Time: 7:30 PM

Location: Online

Chair: Ettore Vescio

Minutes recorded by: Pradip Pandey

Attendees:

- Ettore Vescio (Team Leader)
- Christian Finocchiaro
- Thomas Smith
- Pradip Pandey

B.2 Agenda

1. Finalize Use Case/User Story, ERD and Wireframe Diagrams
2. Complete Section 3,4 & 5 of Assignment 2.
3. Work further on Prototype and tech Stacks.

B.3 Discussion

Requirements Baseline Technology Decision Pack section 1,2,3,4,5 discuss and finalize.

Discussion on Updated Project Controls and Risk Register was done.

Feasibility evidence as given a reality check.

Improved Designs and Diagrams were added and finalized.

Progress includes:

- All sections reviewed
- Section 3,4 & 5 completed
- Viewed RFS feed Data
- Google maps with polygon data working from RFS alert feed

B.4 Decisions Made

- Current team roles remain unchanged
- Decided On Using RFS and SES traffic as SES will supersede BOM.
- Assignment 2 sections discussion and decision continues.
- Diagrams finalized.
- Changelog Decided and improved.

B.5 Open Questions

Polygons and Polylines Could stay on same database?

Discussion on Alert Types, fire, weather?

Discussion on JSON Objects decision.

Action Items

Further Work On prototype to make it up and running.

Work On Feasible Frontend and Backend tech stacks.

Should be free resources.

Task	Owner	Due
Add the Sections 3,4,5 in Typst	Ettore	Before next meeting
Go through the A2 sample and give instruction on the progress of tech stack.	Christian	Before next meeting
Finalize the diagrams and update in Miro Board	Thomas	Before next meeting
Complete the Meeting Minutes and add anything that need to be added on A2 sample and organize suggestions on new tech stack.	Pradip	Before next meeting
Collect Evidence and attach it to section 6 of A2	Ettore	Before A2 submission.

B.6 Next Meeting

Date: Sunday, May 10, 2026

Time: 7:30 PM

B.7 /A2-images/meeting-minutes/meeting-07.md

C Meeting 7 – A2 Work through

C.1 Date & Attendees

Date: Friday, 09 May, 2026

Time: 8:30 PM

Location: Online

Chair: Ettore Vescio

Minutes recorded by: Christian Finocchiaro

Attendees:

- Ettore Vescio (Team Leader)
- Christian Finocchiaro
- Thomas Smith
- Pradip Pandey

C.2 Agenda

1. Present assigned tasks from last meeting
2. Finalize Section 1-5 of Assignment 2
3. Determine any questions to ask Jason during next class final class before submission date
4. Begin preparing for commencement of prototype development

C.3 Discussion

- Section 2 diagrams it was determined some revisions are needed
- SQLite as an alternative to avoid one person hosting the DB
- Google Maps API for map generation, including weaknesses of using the free version
- Data safety, specifically email storage and potential solutions
- Minimising DDoS risk via IP address rate limiting

C.4 Decisions Made

- Google Maps API confirmed for map generation
- SQL lite will be implemented for our database
- The system will be made available to users and run locally not hosted on a web server

C.5 Open Questions

1. Will CSU hypothetically host the final build on the web?
2. In semester 2 will there be a real server available to host the system?
3. Clarify the scenario
 - what is happening in the story and what is the team required to build?

Action Items

1. Pradip
 - Finalise diagram 2 and its description
2. Christian
 - Discuss screen real estate and layout as part of testing 4a
3. Whole team
 1. Finalise individual contributions and update section 6 accordingly
 2. Review Miro board content and flag any changes/ concerns via discord
 3. Add content to Typst and post updates via discord for team review before submission

- 4. Begin working on prototype
- 4. Ettore
 - Finalize Assignment 2 and submit

C.6 Next Meeting

Date: Sunday, May 17, 2026

Time: 7:30 PM